

Overview/Background

Splitting bills among friends, whether at a restaurant, during a group trip, or for shared subscriptions, often involves one person fronting the payment while others pay them back afterward. This system is flawed as it relies on someone being able to front the payment, which can be challenging depending on account balances, credit limits, and/or how large the bill is. Additionally, you rely on those in your group remembering to send the money in a timely manner; if they don't, the one who fronted the payment is put in the awkward position of reaching out to ask for the money. Our app addresses this problem by allowing users to create a group payment session in which all members must approve their share of the cost in real time and send payment before the transaction is processed.

The goal of this project is to build a prototype group payment app that enables one-time payments with real-time consent from all involved parties. The app will support group creation, QR/link-based invitations, and an interface for each user to view and accept their portion of the payment. Only when all users have accepted will a one-time-use credit card be generated for completing the transaction. The final product will be a mobile-first web app built using a React Native frontend and Express.js backend.

Technical Problem Statement

The main technical challenge of this project will be implementing a reliable, real-time group agreement system for payments. This requires the backend to continuously track and reflect the status of each group member's consent and update all user interfaces accordingly. Once full consent is achieved, a simulated one-time-use credit card must be generated for payment completion. The system must also handle edge cases, including, but not limited to, members leaving the group or joining after other members have agreed.

Design Approach

User Interaction:

Main user Flow → Two users (group creator, group joiner)

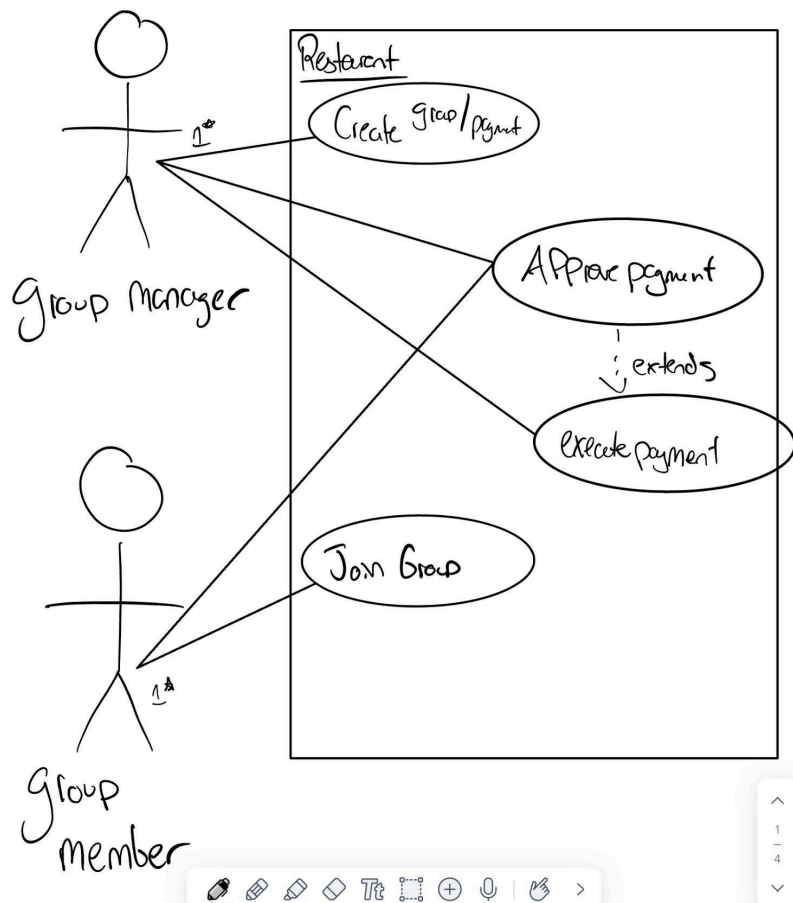
- -> Create Group -> Enter Name + (Enter Bill Amount / Scan Bill)
 - > creates a QR code for others to scan (becomes collapsable once someone joins)
 - > As people join, you see their names and how much is owed for each
 - > # of people taken from how many join the group
 - > Approve button: Individuals approve how much they pay
- > Join Group -> opens camera to scan QR code
 - > See the live amount of how much they would pay

- > Agree button (gets reset to not agree whenever a change is made to the price)
- > Group leader: approves transaction, confirms total amount, pays

Use Case Diagrams:

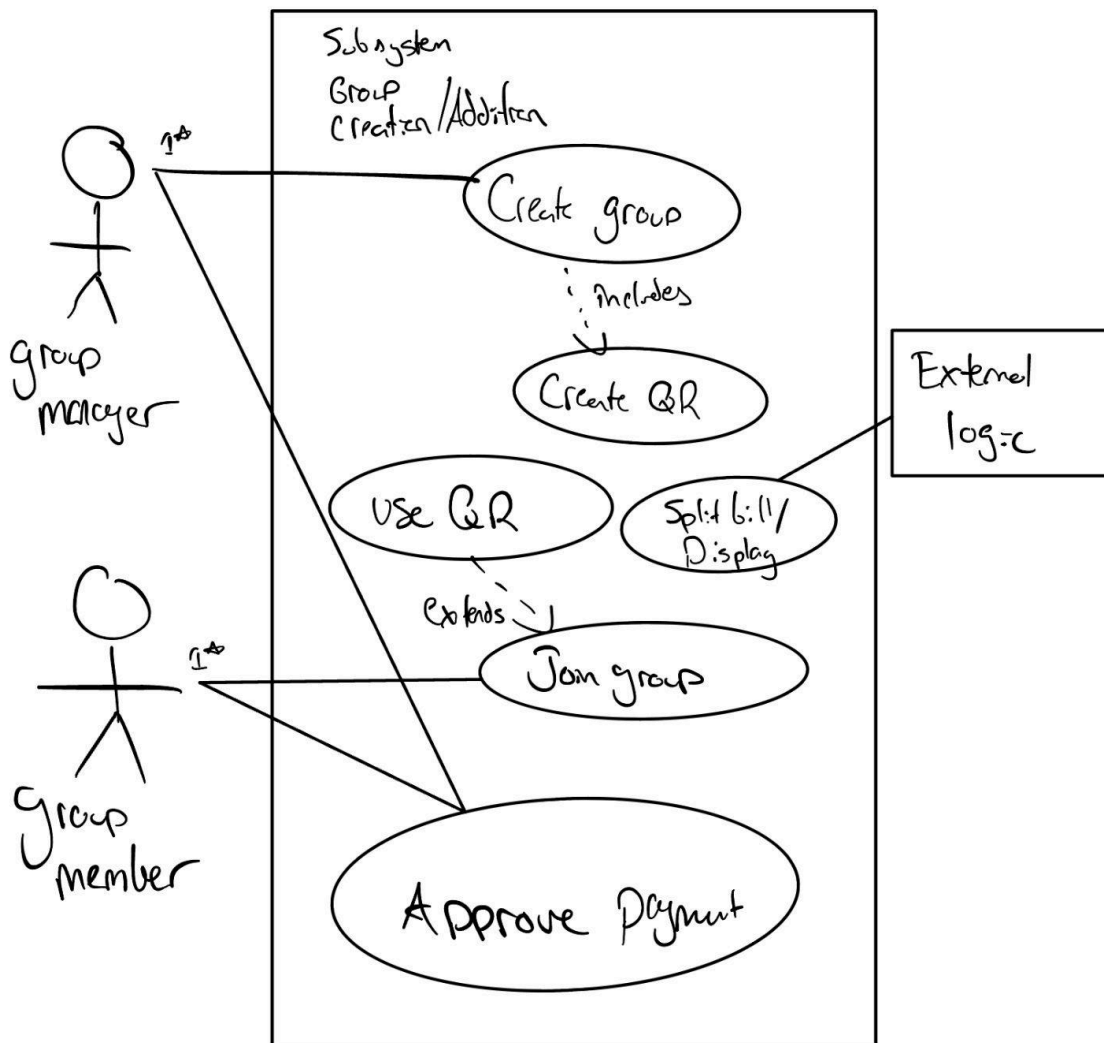
Business:

Business use case diagram

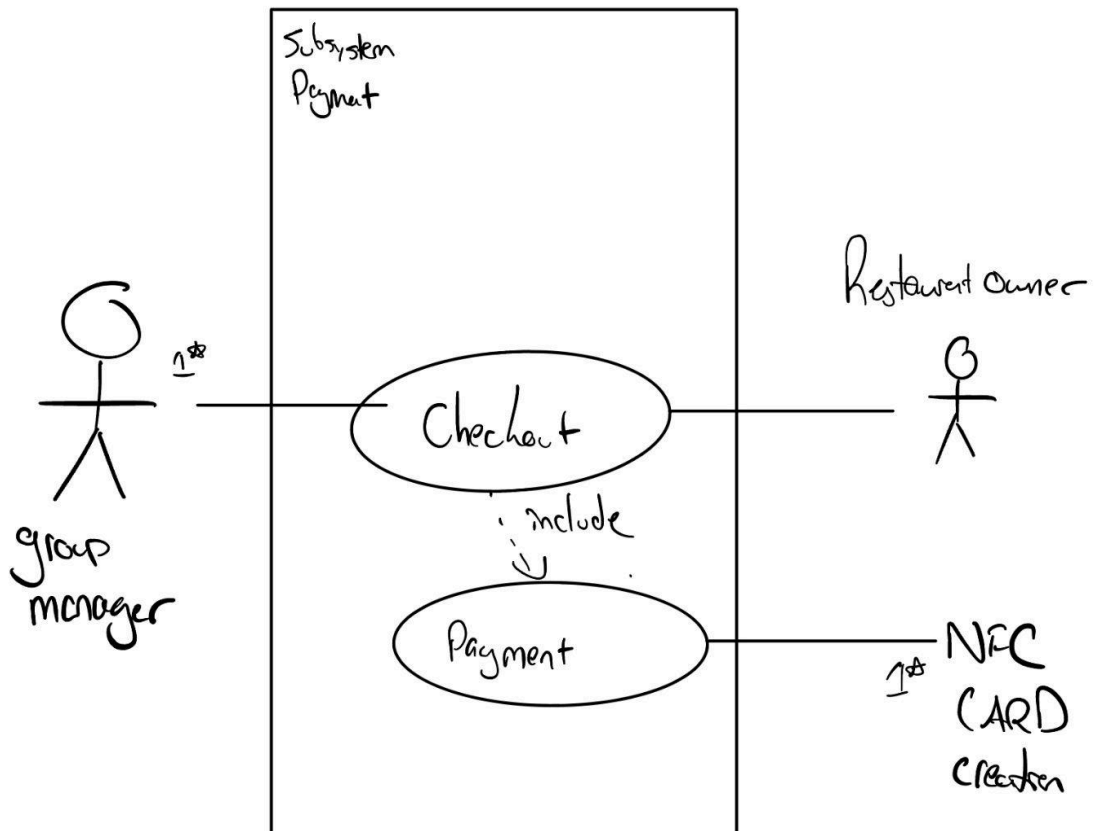


Systems: Group Management

Systems → Group Management, & Payment



Systems: Payment



Tech Stack

Front End

We have chosen to build the Front End using React Native for a variety of reasons. First off, React Native is known to have a good “native look” with mobile applications, which is what we were looking to build. Additionally, React Native is in JavaScript, which our entire team was comfortable with. From an under the hood standpoint, React Native also renders code through diffing the Virtual DOM, which is different from the other presented languages, but the rendering/debugging framework we are familiar with from class. Lastly, out of the languages presented, React Native seems to have the highest popularity/industry adoption, which was a huge draw for us.

We also considered Flutter, Svelte, and Solid. Some of the main drawbacks for why we didn't choose these particular FE frameworks surrounded a lack of a VDOM for rendering, having to learn new languages, the popularity of the framework, or if the advantages of the framework did not align with our project goals as much (in the case of Flutter). Below is the list of pros and cons we considered for each language:

- Flutter:
 - Pros: Works across mobile/desktop/web, has built-in widgets (material) – works well for advanced animations/super customized UI, solid native API integration → no VDOM
 - Cons: New language (Dart), but allegedly really easy to use/learn
- React Native:
 - Pros: JS, good Android/iOS integration, incredibly popular/high industry adoption → has VDOM
 - Cons: Predominantly mobile focused, has performance overhead with DOM diffing
- Svelte:
 - Pros: Primarily used for web-apps, JS/TypeScript, only web-native, rising in popularity but not currently popular
 - Cons: Compiler first → no VDOM. Also, advantageous on static-generation sites.
- SolidJS:
 - Also no VDOM, and web-native, JS/TS, also not as well known

Back End

We have chosen to build the back end using Express.js. Similarly to our reasoning with the frontend, we were drawn by the language being JavaScript. Not only was our team familiar with it, but we also found value in the front end and back end being in the same language.

We also liked that it was so well integrated into the Node.js landscape and that it was commonly used, which would give us applicable experience for future projects.

We also considered Ruby on Rails, FastAPI, and Flask. Some of the main drawbacks for why we didn't choose these particular BE frameworks included having conflicting languages with the FE and having steeper learning curves. Below is the list of pros and cons we considered for each language:

- Ruby on Rails
 - Fast development once comfortable, but has a learning curve, uses ruby as a coding language, MVC, convention over configuration, rigid conventions which can aid in development
- FastAPI
 - Makes developing APIs very fast, python based, really simple data validation
- Express.js
 - Really fast development time, well integrated into the node modules landscape (one of the most common backends in Node.js), good experience to have going forward. Uses JavaScript.
- Flask
 - Python back end, works for web apps and APIs, microframework

App Components to Develop

- Forms
 - Sign Up
 - Login
 - Credit Card Information
- Live user agreement
- Total/split bill costs:
 - How much each person is paying (how many people in the group, and the split
 - Who has accepted/has the owner approved
- NFC Card Generation

Implementing Real Time Aspects

We ultimately decided to use Web Sockets to implement the real time aspects of this app, as it gives us the flexibility and freedom to define our own functions to update our app based on the data being input. Additionally, because we will be using GraphQL in project 2, we found value in gaining experience in a new tool. Below were a few of the points we considered when comparing GraphQL and webhooks.

- GraphQL Subscriptions
 - Version of websockets
 - Better if you already are using GraphQL – lets users subscribe to data and updates instead of defining new functions
 - Will already be using in project 2
- Web Sockets
 - Manually creates functions that provide live updates on data

Data – SQL vs noSQL:

- SQL: good for clearly structured and consistent data
- noSQL: good for complex or inconsistent data

Ultimately, we decided to go with storing our data in a SQL database. This is predominantly because when thinking about the data we are going to store, our data will be structured and consistent. We plan to store user details (name, username, password, email, credit card(s), unique user ID). We decided against using noSQL because for now, our data is not really unstructured. However, in the future we could see the necessity to switch to a noSQL db, if we decide to integrate a receipt upload functionality and store those uploaded receipts for the users to reference later.